

05 Reference

Interview Prep Schedule: June 10 → June 22 (12 Days)

Daily Goal

Rewrite every algorithm from memory. No peeking at `core_algorithms/` until after.

Cumulative Daily Revision (Old + New Every Day)

Date	Day	Rewrite (Old)	New Material	Target Time
Jun 10	0	—	Linear Regression (multi)	10 min
Jun 11	1	Linear Regression	Logistic Regression (multi)	15 min
Jun 12	2	Linear + Logistic Regression	KNN + K-Means	25 min
Jun 13	3	Linear + Logistic + KNN + K-Means	Softmax + Cross Entropy	30 min
Jun 14	4	All above	Neural Network Forward Pass + ReLU	35 min
Jun 15	5	All above	Backpropagation	40 min
Jun 16	6	All above	Metrics + Regularization + Data Splits	45 min
Jun 17	7	Everything	Weak-area drill	45 min
Jun 18	8	Everything	Weak-area drill	45 min
Jun 19	9	Everything	Mock interview pace drill	60 min
	10	Everything	Weak-area drill	45 min

Date	Day	Rewrite (Old)	New Material	Target Time
Jun 20				
Jun 21	11	Everything	Final light review + oral drill	30 min
Jun 22	—	Interview Day	—	—

Folder Map

- `revision/` — rewrite core algorithms here daily
- `day1_knn_kmeans_softmax/` — KNN, K-Means, Softmax, Cross Entropy
- `day2_neural_networks/` — Neural Net, ReLU, Backprop
- `day3_metrics_regularization/` — Metrics, Bias/Variance, Regularization, Data Splits

Oral Drill Checklist (Do While Rewriting)

- What problem does it solve?
- Prediction equation
- Loss function
- Why this loss?
- Gradient intuition
- Training loop
- Inference process
- Common limitations

Mock Interview Readiness

Implement From Memory

- [] Linear Regression
- [] Logistic Regression
- [] KNN
- [] K-Means
- [] Softmax

- [] Multiclass Cross Entropy
- [] Single Neuron Forward Pass

Explain From First Principles

- [] Gradient Descent
- [] MSE
- [] Cross Entropy
- [] Log Odds
- [] Overfitting
- [] Precision / Recall
- [] Backpropagation

Google ML Interview Preparation Tracker

Daily Ritual (Every Morning)

Reimplement from memory. No notes.

Core Algorithms

- [x] Linear Regression (Multi Feature)
- [x] Logistic Regression (Multi Feature)

Target:

- Linear Regression: < 5 min
- Logistic Regression: < 10 min

Oral Drill

For every algorithm:

- What problem does it solve?
 - Prediction equation
 - Loss function
 - Why this loss?
 - Gradient intuition
 - Training loop
 - Inference process
 - Common limitations
-

Day 1

KNN

Theory

- ☐ Intuition
- ☐ Distance Metrics
- ☐ Choosing K
- ☐ Curse of Dimensionality

Implementation

- ☐ Euclidean Distance
- ☐ Neighbor Search
- ☐ Majority Voting
- ☐ Full KNN from Scratch

Oral Drill

- ☐ Why is training fast?
 - ☐ Why is inference slow?
 - ☐ What happens when $K=1$?
 - ☐ What happens when K is very large?
-

K-Means

Theory

- ☐ Clustering Objective
- ☐ Centroids
- ☐ Convergence
- ☐ Choosing K

Implementation

- ☐ Distance to Centroids
- ☐ Cluster Assignment
- ☐ Centroid Update

- ☐ Full K-Means from Scratch

Oral Drill

- ☐ Why does K-Means converge?
 - ☐ Sensitivity to initialization?
 - ☐ What if clusters are non-spherical?
-

Softmax

Theory

- ☐ Why sigmoid is insufficient for multiclass classification
- ☐ Probability interpretation
- ☐ Numerical stability

Implementation

- ☐ Softmax Function
- ☐ Class Prediction

Oral Drill

- ☐ Softmax vs Sigmoid
 - ☐ Why exponentials?
-

Multiclass Cross Entropy

Theory

- ☐ Intuition
- ☐ Relation to Binary Cross Entropy

Implementation

- ☐ Loss Computation

Oral Drill

- ☐ Why not MSE?
 - ☐ What happens if model is confidently wrong?
-

Day 2

Neural Networks

Theory

- ☐ Neuron
- ☐ Layers
- ☐ Hidden Layers
- ☐ Forward Pass

Implementation

- ☐ Single Neuron
- ☐ Dense Layer
- ☐ Multi-Layer Forward Pass

Oral Drill

- ☐ Why are neural networks more expressive than logistic regression?
 - ☐ What is a hidden layer learning?
-

ReLU

Theory

- ☐ Why ReLU?
- ☐ Vanishing Gradient Intuition

Implementation

- ☐ ReLU Activation

Oral Drill

- ☐ ReLU vs Sigmoid
 - ☐ Dead ReLU Problem
-

Backpropagation

Theory

- ☐ Chain Rule
- ☐ Error Propagation

Oral Drill

- ☐ Why does backprop work?
 - ☐ Why chain rule?
-

Day 3

Metrics

- ☐ Accuracy
- ☐ Precision
- ☐ Recall
- ☐ F1 Score
- ☐ ROC-AUC
- ☐ Confusion Matrix

Oral Drill

- ☐ Precision vs Recall
 - ☐ When is Accuracy misleading?
-

Bias / Variance

- ☐ Underfitting
- ☐ Overfitting
- ☐ Bias-Variance Tradeoff

Oral Drill

- ☐ High Bias Symptoms
 - ☐ High Variance Symptoms
-

Regularization

- ☐ L1
- ☐ L2
- ☐ Weight Decay

Oral Drill

- ☐ Why large weights are dangerous?
 - ☐ L1 vs L2
-

Data Splits

- [] Train Set
- [] Validation Set
- [] Test Set

Oral Drill

- [] Why not tune on test data?
-

Mock Interview Readiness

Implement From Memory

- ☐ Linear Regression
- ☐ Logistic Regression
- ☐ KNN
- ☐ K-Means
- ☐ Softmax
- ☐ Multiclass Cross Entropy
- ☐ Single Neuron Forward Pass

Explain From First Principles

- ☐ Gradient Descent
- ☐ MSE
- ☐ Cross Entropy
- ☐ Log Odds
- ☐ Overfitting
- ☐ Precision / Recall
- ☐ Backpropagation

Revision Dataset Philosophy

Every dataset should satisfy:

Rule 1: You can predict the answer before running code.

Rule 2: You can calculate one iteration by hand if needed.

Rule 3: Dataset fits on one screen.

Rule 4: Expected output is obvious.

1. Linear Regression (Single)

Goal: Learn prediction, MSE, gradient descent.

```
X = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11])
```

True relationship: $y = 2x + 1$

Expected: $w \approx 2$, $b \approx 1$

2. Multi-Feature Linear Regression

Goal: Learn $X @ w$, $X.T @ error$.

```
X = np.array([
    [1, 2],
    [2, 1],
    [3, 3],
    [4, 2],
    [5, 4]
])
y = np.array([9, 8, 16, 15, 23])
```

True relationship: $y = 2*x_1 + 3*x_2 + 1$

Expected: $w \approx [2, 3]$, $b \approx 1$

3. Logistic Regression (Single)

Goal: Learn sigmoid, cross entropy, classification.

```
X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([0, 0, 0, 1, 1, 1])
```

Expected: small $X \rightarrow$ class 0, large $X \rightarrow$ class 1. Weight positive.

4. Multi-Feature Logistic Regression

```
X = np.array([
    [1, 1],
    [1, 2],
    [2, 1],
    [3, 3],
    [4, 3],
    [4, 4]
])
y = np.array([0, 0, 0, 1, 1, 1])
```

Expected: Both features increase with class label. $w_1 > 0$, $w_2 > 0$, $b < 0$.

5. K-Means

```
X = np.array([
    [1, 1],
    [1, 2],
    [2, 1],
    [10, 10],
    [10, 11],
    [11, 10],
    [20, 20],
    [20, 21],
    [21, 20]
])
K = 3
```

Expected centroids: $[-1.33, -1.33]$, $[-10.33, -10.33]$, $[-20.33, -20.33]$

6. KNN

```
X_train = np.array([
    [1, 1],
    [2, 1],
    [10, 10],
    [11, 10]
])
y_train = np.array([0, 0, 1, 1])

x_new_1 = np.array([1.5, 1.5]) # Expected: Class 0
x_new_2 = np.array([10.5, 10.5]) # Expected: Class 1
```

7. Softmax

3 classes.

```
X = np.array([
    [1, 1],
    [2, 1],
    [10, 10],
    [11, 10],
    [20, 20],
    [21, 20]
])
y = np.array([0, 0, 1, 1, 2, 2])
```

Expected: small → class 0, medium → class 1, large → class 2.

8. Neural Network

Reuse Softmax dataset. One hidden layer.

9. Metrics

```
y_true = np.array([0, 0, 0, 1, 1, 1])
y_pred = np.array([0, 0, 1, 1, 1, 0])
```

Use for: Accuracy, Precision, Recall, F1, Confusion Matrix.

10. Bias-Variance

```
X = np.array([1, 2, 3, 4, 5])
y = np.array([1, 4, 9, 16, 25])
```

True relationship: $y = x^2$

Use for: underfitting, overfitting, bias, variance discussion.

Summary

Every dataset is:

- tiny
- deterministic
- visually obvious
- easy to debug
- easy to explain in an interview